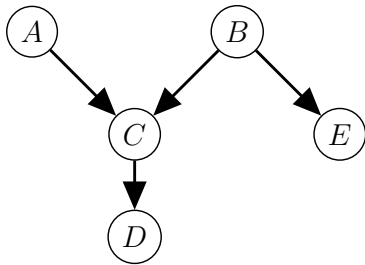

University of Waterloo
School of Computer Science
CS 486/686, **SAMPLE** Final Examination
Winter 2025

- Instructor: Jesse Hoey and Victor Zhong
- Date: Sometime Before April 22nd, 9:00am
- Location: Wherever
- Time: Whenever
- There are 5 questions on this exam
- There are 6 pages in this exam
- There are a total of 100 marks on the exam
- You have 150 minutes to complete the exam
- Non-programmable calculators are allowed
- Write your answers on a separate sheet. On the **REAL** final, you will write directly on the exam (space will be provided)

Question 1: Short Answers (10 points out of a total of 100 points)

- (1a) [2 points] In Assignment 3, you used maximum likelihood to learn the parameters of a Bayesian network. Explain what you would need to do to use Maximum A-posteriori (MAP) learning for the Naive Bayes model instead. Be brief.

Consider this Bayesian Network and answer the following two questions about it



- (1b) [2 points] In the BN shown above, is A conditionally independent of B given D ? Simply answer True or False.
- (1c) [2 points] In the BN shown above, is E conditionally independent of D given B ? Simply answer True or False.
- (1d) [2 points] A Bayesian reasoner can provide an estimate of the probability distribution over some random variable before it has observed any evidence of that random variable. A frequentist reasoner cannot do this. In a *single sentence* explain why.
- (1e) [2 points] Fill in the blank: A statement B is a *logical consequence* of a set of statements A if _____.

Question 2: Search (25 points out of a total of 100 points)

Consider the following puzzle which consists of a sequence of seven squares. In the squares are three black tiles (B) and three white tiles (W). The remaining square is empty (E). The rules and costs, $g(n)$, are as follows:

- A tile may move into an adjacent empty square with a cost of one.
- A tile may hop over at most two other tiles into an empty square with a cost equal to the number of tiles hopped over plus one.

The goal of the puzzle is to get all of the black tiles to be to the right of all of the white tiles (without regard to the location of the empty square) using a sequence of moves with the lowest possible cost.

Let $h(n)$ (the heuristic) be the number of white tiles which have one or more black tiles to the left (i.e., between the white tile and the leftmost edge of the puzzle, there is at least one black tile) *plus* the number of black tiles which have one or more white tiles to the right (i.e., between the black tile and the rightmost edge of the puzzle, there is at least one white tile). For example, applying $h(n)$ to the configuration below gives $2 + 3 = 5$.

W	B	B	B	W	W	E
---	---	---	---	---	---	---

Break ties by expanding the node with the smallest $h(n)$ value, according to the smallest cost ($g(n)$) if all $h(n)$ values are the same and then as the node where the “E” square is furthest to the left if both $g(n)$ and $h(n)$ are the same. That is, if $g(n)$ and $h(n)$ are the same, then expand the node having the leftmost “E” square among all the nodes

(2a) [5 points]

Is the $h(n)$ defined above admissible? Carefully explain why or why not.

(2b) [10 points]

Show part of the search graph that results from applying algorithm A* starting from the configuration shown above (WBBBWWE) with the heuristic $h(n)$ and cost $g(n)$ defined above. Specifically, continue until you have expanded (generated the successors of) three nodes. Number the three nodes you expand to indicate the order in which they are expanded (“1”, “2”, “3”). Then, number (with “4”) the next leaf that you would expand if you were to continue with A* search on this graph. Label each node with its $g(n)$ value and its $h(n)$ value. You should label all nodes, not just the ones you expand. Do not add a node to the tree if it is already in the tree somewhere with a lower or equal value of $f(n) = g(n) + h(n)$.

(2c) [**10 points**]

Do the same as in part (b) of this question, but this time use **heuristic depth-first search** and only expand 2 nodes (you will therefore number 3 nodes). Label all nodes as in part (b).

Question 3: Reinforcement Learning (15 points out of a total of 100 points)

A robot can be in one of 4 positions (s_1, s_2, s_3, s_4) in a corridor. At the four positions, the robot gets a reward of $(0, 0, +3, -1)$ for states (s_1, s_2, s_3, s_4) respectively. The robot can move left or right, and its movements are successful with probability 0.8 (with probability 0.1 it moves in the opposite direction and with probability 0.1 it stays in the same spot). Note that it cannot choose to stay in the same spot, it must try to move either left or right. The corridor is circular, so s_4 is to the left of s_1 and s_1 is to the right of s_4 . The Q-value for the robot is initially as follows

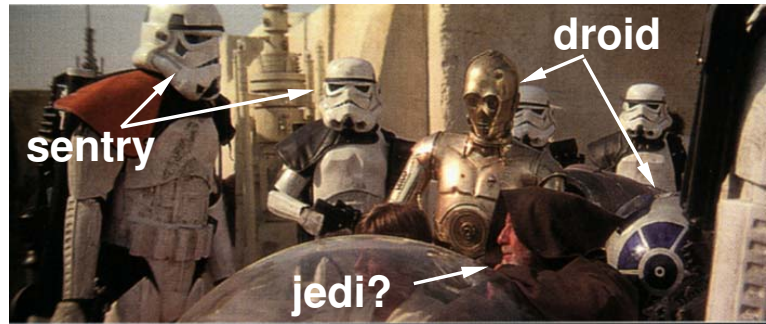
Q(s,a)	state			
action	s_1	s_2	s_3	s_4
right	15	10	30	5
left	20	25	10	0

Using a discount factor of $\gamma = 0.9$ and a learning rate of $\alpha = 0.5$, use Q-learning to update the $Q(s, a)$ function for 3 iterations, given the robot starts in state s_1 , and acts greedily with respect to its current Q values, but that the first action it takes fails (and the robot stays stationary). Assume the other two actions succeed. Assume the reward is gathered from a state when the agent takes an action in that state (regardless of whether the action succeeds or not). Show clearly the state s , the action taken a , the reward obtained, r , and the new state s' , as well as the $Q(s, a)$ function in a table like the one above after each iteration. Show your work for updating each time.

Recall that a Q-learning agent updates $Q(s, a)$ after each experience tuple $\{s, a, r, s'\}$ as a weighted sum of its current estimate and the estimate from the experience given by $r + \gamma \max_{a'} Q(s', a')$, with weights given by $(1 - \alpha)$ and α , respectively.

Question 4: Bayesian Networks (25 points out of a total of 100 points)

A pair of sentry (guard) robots are looking for a set of droids (other robots). They stop each group of travellers (which always contain some droids and some humans) and try to detect if the group includes the droids they are looking for. There is only a 10% chance that any given group will contain the droids they are looking for. It is also known that the



droids they are looking for travel often (80% of the time) with a powerful human called a “Jedi”. Other droids only travel with Jedi 1% of the time. The sentry robots each will detect the droids that they are looking for with probability 0.9 if they are actually in the group, and with probability 0.01 if they are not in the group (a false alarm). However, Jedi will always perform a “mind trick” when they meet sentry robots which makes the sentries always fail to detect the droids (whether they are present or not).

(5a) [10 points]

Draw a Bayesian network representing this domain. Clearly label all your variables and write down the conditional probability tables.

(5b) [10 points]

Suppose that one particular sentry robot detects the droids they are looking for, but the other does not. Compute the probability that the group includes the droids that the sentries are looking for.

(5c) [2 points]

Now suppose a third sentry robot joins the other two. It has the same probabilities of detecting droids as the other two. It’s battery is dead though and it shuts itself off to recharge, so can’t sense anything. Now compute the probability that the group includes the droids that the sentries are looking for.

(5d) [3 points] Now this third robot wakes up (fully charged) and detects that the droids are in the group (so now you have two sentry robots detecting that the droids are there, and one that does not). Now compute the probability that the group includes the droids that the sentries are looking for.

Question 5: Decision Networks (25 points out of a total of 100 points)

When driving a long distance in Canada, the probability of having an accident is 0.2 if it is not snowing and 0.6 if it is snowing. It is snowing 30% of the time in Canada. There are two things drivers can do to help themselves:

- they can *chain-up* (put chains on the tires to increase traction), which halves the probability of having an accident if it is snowing (and has no effect if it is not snowing), or
- they can *sleep*, which halves the probability of having an accident regardless of the weather.

If a person chains-up and sleeps, the probability of having an accident is the product of both effects, so is decreased by a factor of 4. However, sleeping and chaining up cost 20 units each, and having an accident costs 200. If a person sleeps and chains-up, it costs 40.

(6a) [5 points]

Draw a decision network to represent this problem. Show all conditional probability tables, and show the utility function. Assume the decisions are made based on whether it is snowing or not, and that the decision to *chain-up* is made first (so the decision to *sleep* will be based upon the decision to *chain-up*).

(6b) [20 points]

Construct the optimal decision functions for both decisions and the optimal value for the entire network. Do this by first eliminating any variables that have no decision nodes as children. Then, eliminate the decisions, and finally eliminate any remaining variables.